



ESCUELA
COLOMBIANA
DE INGENIERÍA
JULIO GARAVITO

VIGILADA MINEDUCACIÓN

UNIVERSIDAD

Bitácora Bash - Powershell

Juan Esteban Sánchez García

Escuela Colombiana de Ingeniería Julio Garavito

ODSC LAB

Docente: Yeison Steven Barreto Rodríguez

Bogotá, Colombia

LINUX - BASH

PUNTO 1 - HELLO WORLD

```
#!/bin/bash
clear
echo "Hello World from Shell"
```

Para ejecutarlo:

chmod +x 1-hello.sh

./1-hello.sh

PUNTO 2

```
#!/bin/bash
clear

line_count=$(wc -l < /etc/profile)
echo "The number of lines on /etc/profile is: $line_count"
```

Para ejecutarlo:

chmod +x 2-sequence.sh

./2-sequence.sh

PUNTO 3

```
#!/bin/bash
clear

if [ "$#" -ne 2 ]; then
    echo "Usage: $0 word file"
    echo "Example: $0 house /etc/passwd"
    exit 1
fi

word=$1
file=$2

if [ ! -f "$file" ]; then
    echo "The file '$file' does not exist."
    exit 1
fi

echo "Searching for '$word' in '$file':"
```

```
grep -n -- "$word" "$file"
```

Para ejecutarlo:

```
chmod +x search_word.sh
```

```
./search_word.sh casa /etc/passwd
```

PUNTO 4

```
#!/bin/bash
clear

output_file="users_extraction.txt"

echo "Extracting usernames and descriptions from /etc/passwd..."
> "$output_file"

while IFS=':' read -r username _ _ _ description _; do

    [ -z "$username" ] && continue
    echo "$username: $description" >> "$output_file"
done < /etc/passwd

echo "Result saved to: $output_file"
```

Para crear usuarios:

```
sudo useradd -m -c "First user" User1
```

```
sudo useradd -m -c "Second user" User2
```

```
sudo useradd -m -c "Third user" User3
```

```
sudo useradd -m -c "Fourth user" User4
```

```
sudo useradd -m -c "Fifth user" User5
```

Para ejecutarlo:

```
chmod +x 4-extract_users.sh
```

```
./4-extract_users.sh
```

```
cat users_extraction.txt
```

PUNTO 5

```
#!/bin/bash
clear

if [ "$#" -ne 2 ]; then
    echo "Usage: $0 directory permission_string"
    echo "Example: $0 /etc/ -rw-r--r--"
    exit 1
fi

dir=$1
perm_string=$2

if [ ! -d "$dir" ]; then
    echo "Error: '$dir' is not a valid directory."
    exit 1
fi

echo "Searching in '$dir' for files with permissions '$perm_string'..."

perm_numeric=$(stat -c '%a' /dev/null 2>/dev/null)

find "$dir" -maxdepth 1 -type f -printf '%M %p\n' | awk -v
p="$perm_string" '$1 == p'
```

Al listar las características de un archivo con el comando `ls -l`, el campo de permisos está compuesto por 10 caracteres, ¿Que significan cada uno de ellos? y qué valores pueden tomar?

Los 10 caracteres se interpretan así:

1. Carácter 1: tipo de archivo.

- - archivo regular
- **d** directorio
- **l** enlace simbólico
- **c** dispositivo de caracteres
- **b** dispositivo de bloques
- **p** FIFO (pipe con nombre)

- **s** socket
2. Caracteres 2–4: permisos del propietario (user).
 - **r** lectura, - sin lectura
 - **w** escritura, - sin escritura
 - **x** ejecución, - sin ejecución
 - En el tercer lugar también pueden aparecer **s/S** (setuid).
 3. Caracteres 5–7: permisos del grupo (group).
 - Mismo esquema **rwX** o **-**.
 - En el tercer lugar pueden aparecer **s/S** (setgid).
 4. Caracteres 8–10: permisos de otros (others).
 - Mismo esquema **rwX** o **-**.
 - En el último lugar pueden aparecer **t/T** (sticky bit).

Ejemplo: **-rw-r--r--**

- **-** archivo regular.
- **rw-** owner: leer y escribir.
- **r--** group: solo leer.
- **r--** others: solo leer.

Para ejecutarlo:

`chmod +x 5-find_files.sh`

`./5-find_files.sh /etc/ -rw-r--r-`

PUNTO 6

```
#!/bin/bash

while true; do
    clear
    echo "=====
    echo "          MAIN MENU"
    echo "=====
    echo "1) Hello World"
    echo "2) Count lines in /etc/profile"
    echo "3) Search word in file"
    echo "4) Extract users from /etc/passwd"
    echo "5) Find files by permissions"
    echo "6) Exit"
    echo "=====
    read -r -p "Choose an option (1-6): " option
```

```

case "$option" in
    1)
        ./1-hello.sh
        read -r -p "Press Enter to continue..." _
        ;;
    2)
        ./2-sequence.sh
        read -r -p "Press Enter to continue..." _
        ;;
    3)
        read -r -p "Enter word to search: " word
        read -r -p "Enter file path: " file
        ./3-search_word.sh "$word" "$file"
        read -r -p "Press Enter to continue..." _
        ;;
    4)
        ./4-extract_users.sh
        read -r -p "Press Enter to continue..." _
        ;;
    5)
        read -r -p "Enter directory: " dir
        read -r -p "Enter permission string (e.g. -rw-r--r--): " perms
        ./5-find_files.sh "$dir" "$perms"
        read -r -p "Press Enter to continue..." _
        ;;
    6)
        echo "Exiting..."
        exit 0
        ;;
    *)
        echo "Invalid option."
        read -r -p "Press Enter to continue..." _
        ;;
esac
done

```

Para ejecutarlo:

chmod +x menu.sh

./menu.sh

PUNTO 7.1

```
#!/bin/bash
clear

# Validate arguments
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 name directory"
    echo "Example: $0 poem.txt /home/user/Documents"
    exit 1
fi

name=$1
dir=$2
path="$dir/$name"

if [ ! -d "$dir" ]; then
    echo "Error: '$dir' is not a valid directory."
    exit 1
fi

echo "Checking '$name' inside '$dir'..."

if [ -e "$path" ]; then
    if [ -f "$path" ]; then
        echo "'$name' is a regular FILE."
    elif [ -d "$path" ]; then
        echo "'$name' is a SUBDIRECTORY."
    else
        echo "'$name' is SOMETHING ELSE (link, device, socket, etc.)."
    fi
else
    echo "'$name' does not exist in that directory."
fi
```

Para ejecutarlo:

chmod +x 7.1-check_entry.sh

./7.1-check_entry.sh poem.txt

/home/juan/Escritorio/Git-Repos/ODSC/Platforms/Platform\ 3/Linux-Scripts

PUNTO 7.2 - Depende del Linux, este es para Arch Linux

```
#!/bin/bash
clear

echo "Checking illegal access attempts to user root (Arch Linux,
journalctl)..."
echo "=====

entries=$(journalctl -q _UID=0 --no-pager | grep -Ei
"failed|authentication failure|incorrect password")

if [ -z "$entries" ]; then
    echo "No failed attempts to root found."
    exit 0
fi

count=0
while IFS= read -r line; do
    echo "$line"
    count=$((count + 1))
done <<< "$entries"

echo "=====
echo "Total failed attempts to root: $count"
echo "=====
```

Para ejecutarlo:

chmod +x 7.2-root_attempts.sh

sudo ./7.2-root_attempts.sh

WINDOWS - POWERSHELL

PUNTO 1

```
Clear-Host
Write-Host "Hello World from Shell"
```

Para ejecutarlo

.\1-hello.ps1

PUNTO 2

```
Clear-Host
$path = "C:\UNIVERSIDAD-GIT\Proyectos\ODSC\Platforms\Platform
3\Windows-Scripts\Poem\poem.txt"
$lineCount = (Get-Content $path).Length
Write-Host "The number of lines on the poem is: $lineCount"
```

Para ejecutarlo

.\2-secuence.ps1

PUNTO 3

```
param(
    [Parameter(Mandatory = $true)]
    [string]$word,
    [Parameter(Mandatory = $true)]
    [string]$file
)

Clear-Host

if (-not (Test-Path -Path $file -PathType Leaf)) {
    Write-Host "The file '$file' does not exist."
    exit 1
}

Write-Host "Searching for '$word' in '$file':"
$results = Select-String -Path $file -Pattern $word -SimpleMatch

if ($results) {
    foreach ($resultado in $results) {
        Write-Host "Found on line $($resultado.LineNumber):
$($resultado.Line)"
    }
}
```

```

    }
} else {
    Write-Host "The word '$word' was not found in the file."
}

```

Para ejecutarlo

.\3-buscar_palabra.ps1 sangre

"C:\UNIVERSIDAD-GIT\Proyectos\ODSC\Platforms\Platform
3\Windows-Scripts\Poem\poem.txt"

PUNTO 4

```

param(
    [string]$file_exit = "users_extraction.txt"
)

Clear-Host

Write-Host "Extracting system users and descriptions..."
-ForegroundColor Cyan

$users = Get-LocalUser | Where-Object { $_.Enabled -eq $true -and
$_Name -notlike "Default*" -and $_Name -notlike "*$" }

$resultado = @()
foreach ($users in $users) {
    $description = if ($users.Description) { $users.Description } else
{ "No description" }
    $resultado += "$($users.Name): $description"
}

$resultado | Out-File -FilePath $file_exit -Encoding UTF8

Write-Host "Result saved in: $file_exit" -ForegroundColor Green
Write-Host "First 6 lines:" -ForegroundColor Yellow
Get-Content $file_exit -Head 6

```

Para ejecutarlo

.\4-extraer_usuarios.ps1

PUNTO 5

```
param(
    [Parameter(Mandatory = $true)]
    [string]$directory,
    [Parameter(Mandatory = $true)]
    [string]$permit
)

Clear-Host

if (-not (Test-Path -Path $directory -PathType Container)) {
    Write-Host "Error: '$directory' is not a valid directory."
    -ForegroundColor Red
    exit 1
}

Write-Host "Searching for files in '$directory' with permissions '$permit':" -ForegroundColor Cyan

switch ($permit) {
    "\FullControl" { $acRights =
[System.Security.AccessControl.FileSystemRights]::FullControl }
    "\Read" { $acRights =
[System.Security.AccessControl.FileSystemRights]::Read }
    "\Write" { $acRights =
[System.Security.AccessControl.FileSystemRights]::Write }
    "\Execute" { $acRights =
[System.Security.AccessControl.FileSystemRights]::ExecuteFile }
    default {
        Write-Host "Unrecognized permission. Use: \FullControl, \Read,
\Write, \Execute" -ForegroundColor Red
        exit 1
    }
}

Get-ChildItem -Path $directory -File | ForEach-Object {
    try {
```

```

        $acl = Get-Acl -Path $_.FullName
        $access = $acl.Access | Where-Object {
            $_.FileSystemRights -band $acRights -eq $acRights -and
            $_.AccessControlType -eq "Allow"
        }
        if ($access) {
            Write-Host "$($_.Name) - Permission:
$($access.FileSystemRights)" -ForegroundColor Green
        }
    }
    catch {
    }
}

```

Para ejecutarlo

.\5-buscar_archivos.ps1 "C:\Windows\inf" "\FullControl"

PUNTO 6

```

function Clean {
    Clear-Host
}

do {
    Clean
    Write-Host "=====" -ForegroundColor Cyan
    Write-Host "          PRINCIPAL MENU" -ForegroundColor Cyan
    Write-Host "=====" -ForegroundColor Cyan
    Write-Host "1) Hello World"
    Write-Host "2) Count poem lines"
    Write-Host "3) Search word in file"
    Write-Host "4) Extract system users"
    Write-Host "5) Search files by permission"
    Write-Host "6) Exit"
    Write-Host "=====" -ForegroundColor Cyan
    $opcion = Read-Host "Choose an option (1-6)"

    switch ($opcion) {
        "1" {
            Clean
            & ".\1-hello.ps1"
            Read-Host "Press Enter to continue"
        }
    }
}

```

```

    "2" {
        Clean
        & ".\2-secuencia.ps1"
        Read-Host "Press Enter to continue"
    }
    "3" {
        Clean
        $palabra = Read-Host "Enter word to search"
        $archivo = Read-Host "Enter file path"
        & ".\3-buscar_palabra.ps1" $palabra $archivo
        Read-Host "Press Enter to continue"
    }
    "4" {
        Clean
        & ".\4-extraer_usuarios.ps1"
        Read-Host "Press Enter to continue"
    }
    "5" {
        Clean
        $directorio = Read-Host "Enter directory"
        $permiso = Read-Host "Enter permission (\FullControl,
\Read, etc.)"
        & ".\5-buscar_archivos.ps1" $directorio $permiso
        Read-Host "Press Enter to continue"
    }
    "6" {
        Write-Host "Goodbye!" -ForegroundColor Green
        break
    }
    default {
        Write-Host "Invalid option." -ForegroundColor Red
        Read-Host "Press Enter"
    }
}
} while ($opcion -ne "6")

```

Para ejecutarlo

.\6-menu.ps1

PUNTO 7.1

```
param(
    [Parameter(Mandatory=$true)]
    [string]$name,
    [Parameter(Mandatory=$true)]
    [string]$directory
)

Clear-Host

$ruta_completa = Join-Path $directory $name

if (-not (Test-Path $directory -PathType Container)) {
    Write-Host "Error: '$directory' not a valid directory."
    -ForegroundColor Red
    exit 1
}

Write-Host "Verifying '$name' in '$directory':" -ForegroundColor Cyan

if (Test-Path $ruta_completa) {
    $item = Get-Item $ruta_completa
    switch ($item.PSIsContainer) {
        $false { Write-Host "It is a regular FILE" -ForegroundColor
Green }
        $true { Write-Host "It is a SUBdirectory" -ForegroundColor
Green }
    }
    Get-Item $ruta_completa | Format-List
} else {
    Write-Host "Does not exist in the specified directory"
    -ForegroundColor Red
}
```

Para ejecutarlo

.\7.1-verificar_archivo.ps1 "poem.txt"

**"C:\UNIVERSIDAD-GIT\Proyectos\ODSC\Platforms\Platform
3\Windows-Scripts\Poem"**

PUNTO 7.2

```
Clear-Host

$logName = "Security"
$adminName = "Administrator"

Write-Host "Searching for failed logon attempts to $adminName..."
-ForegroundColor Cyan

$failedEvents = Get-WinEvent -FilterHashtable @{ LogName = $logName; ID
= 4625 } -MaxEvents 1000 -ErrorAction SilentlyContinue |
    Where-Object { $_.Message -match $adminName -or $_.Message -match
$env:COMPUTERNAME }

$counter = 0
Write-Host "=====" -ForegroundColor Yellow
Write-Host "Failed logon attempts to $adminName" -ForegroundColor
Yellow

Write-Host "=====" -ForegroundColor Yellow

foreach ($ev in $failedEvents) {
    $time = $ev.TimeCreated.ToString("yyyy-MM-dd HH:mm:ss")
    Write-Host "[$time] $($ev.Id) - $($ev.Message.Split('\n')[0])"
-ForegroundColor Red
    $counter++
}

Write-Host "=====" -ForegroundColor Yellow
Write-Host "TOTAL failed attempts: $counter" -ForegroundColor Yellow
Write-Host "=====" -ForegroundColor Yellow
```

Para ejecutarlo

.\7.2-intentos-admin.ps1